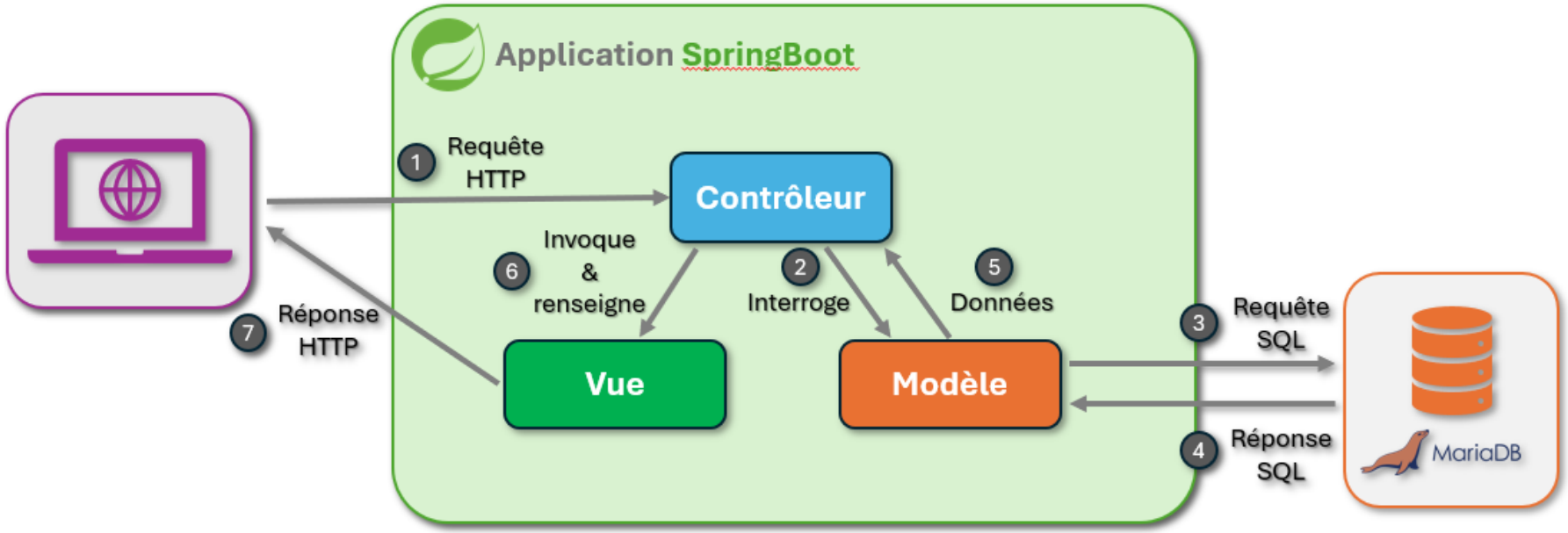
Développer un Template

Introduction au templating

Présentation & Principes de Base

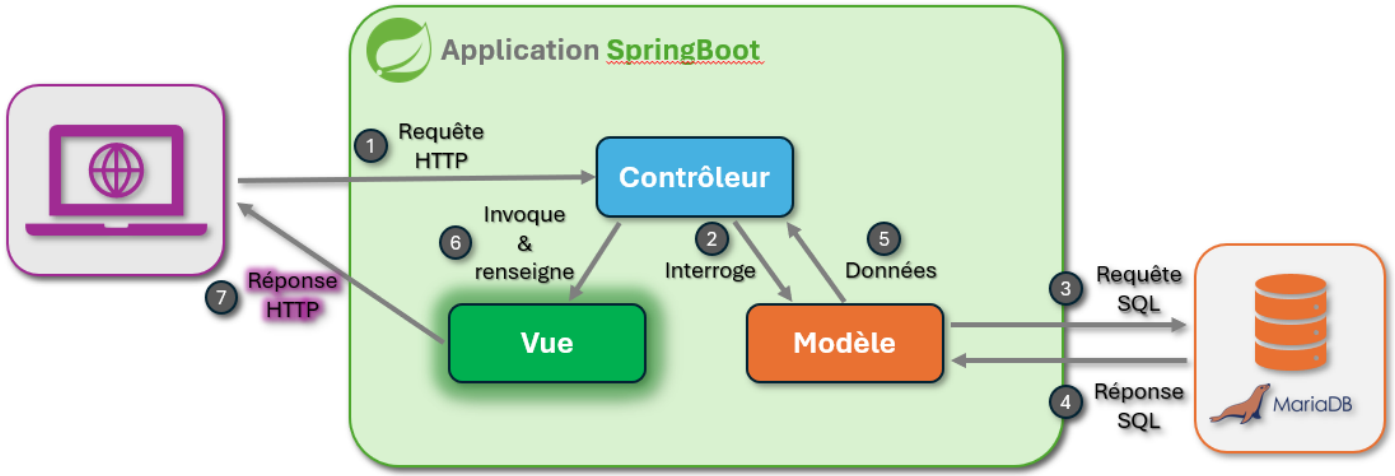
INTRODUCTION AU TEMPLATING

- Nous avons vu que **SpringBoot** propose une architecture **MVC** (**M**odel-**V**ue-**C**ontrôleur) pour développer nos applications webs.



INTRODUCTION AU TEMPLATING

- La **Vue** aura pour mission d'élaborer une réponse à partir des données fournies par le contrôleur.
- La réponse sera au standard **HTTP** afin de transmettre en général une page **HTML**.



INTRODUCTION AU TEMPLATING

- Le templating dans notre contexte est la solution technique qui permettra de construire page Web de réponse en reprenant une page **HTML** enrichie du langage de la solution de templating.

- Le langage de templating permet l'insertion des données fournies par le contrôleur.

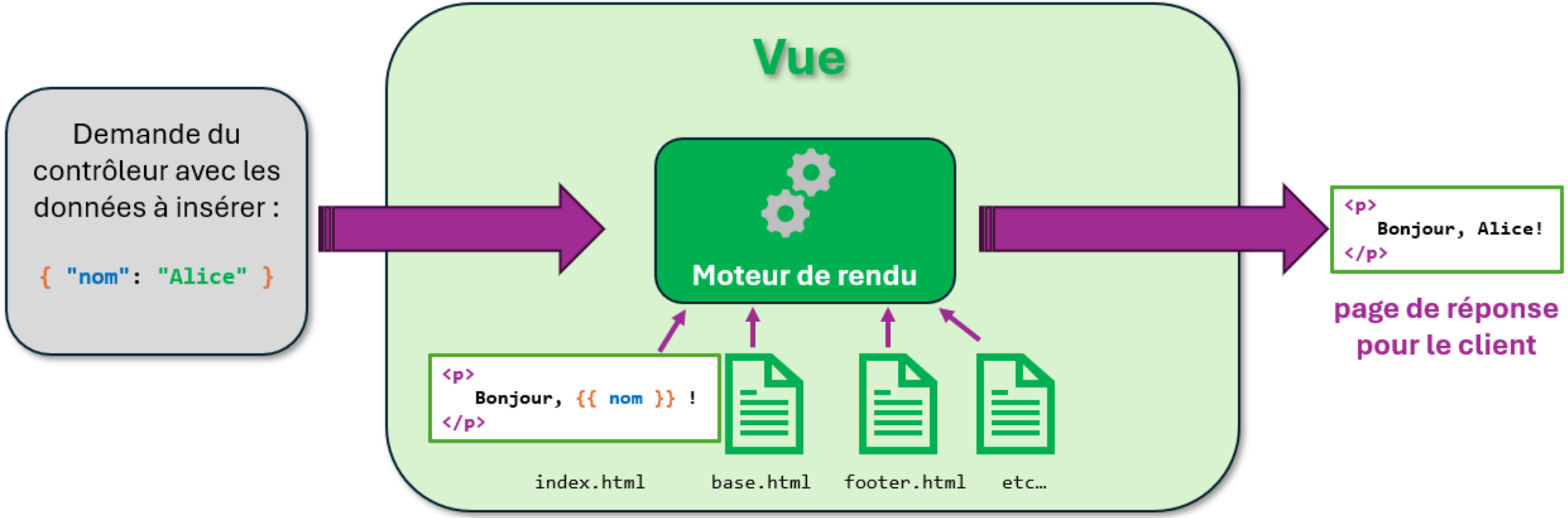
- Le fichier de template est choisi par le contrôleur parmi l'ensemble des fichiers templates rédigés en amont par le développeur.

INTRODUCTION AU TEMPLATING

- **Le moteur de template repose sur 3 éléments fondamentaux :**
  - **Un modèle (template) :**
    → Un ou plusieurs fichiers **HTML** contenant des zones dynamiques à remplacer (variables, boucles, conditions…).
  - **Un contexte (ou modèle de données) :**
    → Une structure (souvent un objet ou une map) contenant les données à injecter dans le template.
  - **Un moteur de rendu :**
    → Le moteur lit le template, remplace les variables par leurs valeurs, évalue les instructions conditionnelles et boucle, puis produit une page **HTML** finale.

INTRODUCTION AU TEMPLATING

- Le schéma ci-dessous illustre le processus avec les éléments de code utilisés par chaque partie (contrôleur; template et résultat)



INTRODUCTION AU TEMPLATING

- **Les moteurs de template les plus courants :**
  - **Jinja2** (Python, Flask)
  - **Twig** (PHP, Symfony)
  - **Mustache / Handlebars** (JavaScript)
  - **EJS / Pug** (Node.js)
  - **Thymeleaf** (Java / Spring Boot)

Particularités de Thymeleaf

- **Naturellement intégrable :**
  - C'est-à-dire que les templates **Thymeleaf** restent **valides et affichables** dans un navigateur, même avant d'être interprétés par le serveur.

- **Syntaxe non intrusive :**
  - Les balises et attributs utilisés par **Thymeleaf** sont **des attributs HTML personnalisés**, comme `th:text`, `th:each`, `th:if`, etc.
    Cela permet de garder des fichiers **HTML lisibles** et **compatibles** avec n'importe quel éditeur ou designer.

Particularités de Thymeleaf

- **Exemple :**
  - `<p th:text="'Bonjour, ' + ${nom} + ' !'">Bonjour, [nom]</p>`
    - **Affiche :** Bonjour, Alice !
    - **Si on ouvre le fichier sans serveur, on voit :** Bonjour, [nom]

Construire son template

Avec Thymeleaf

SYNTAXE DE BASE

- **L'insertion de données :**
  - `${variable}` **:** accède à une **variable** du modèle
  - `@{/url}` **:** résout une **URL** relative à l'application
  - `*{champ}` **:** accède à un **champ** de l'objet lié à un formulaire
  - `#{message}` **:** récupère un **message internationalisé**

Implémentation avec Spring Boot

- **Exemple :**
  - Le contrôleur

```
@Controller
class HelloController {
    @GetMapping("/hello")
    fun hello(model: Model): String {
        model.addAttribute("nom", "Alice")
        return "hello" // correspond à templates/hello.html
    }
}
```

Implémentation avec Spring Boot

- **Exemple :**
  - Le template (`src/templates/hello.html`) :

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <title>Exemple Thymeleaf</title>
</head>
<body>
    <h1 th:text="'Bonjour, ' + ${nom} + ' !'">Bonjour, [nom]</h1>
</body>
</html>
```

Implémentation avec Spring Boot

- **Utilisation des variables :**
  - Les variables **Thymeleaf** proviennent du **modèle (Model)** envoyé depuis le contrôleur **Spring**.

  - <u>Exemple côté contrôleur :</u>

```
@Controller
class DemoController {
    @GetMapping("/demo")
    fun demo(model: Model): String {
        model.addAttribute("nom", "Alice")
        model.addAttribute("age", 22)
        return "demo"
    }
}
```

Implémentation avec Spring Boot

- **Utilisation des variables :**
  - Exemple côté template :

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
  <meta charset="UTF-8">
  <title>Démo Thymeleaf</title>
</head>
<body>
  <p>Nom : <span th:text="${nom}">nom_par_défaut</span></p>
  <p>Âge : <span th:text="${age}">age_par_défaut</span> ans</p>
</body>
</html>
```

Implémentation avec Spring Boot

- **La structure conditionnelle :**
  - Syntaxe :
    - `th:if` affiche l'élément seulement si la condition est vraie.
    - **th:unless** est l'inverse de **th:if**.
  - Exemples :

```
<p th:if="${age >= 18}">Vous êtes majeur.</p>
<p th:unless="${age >= 18}">Vous êtes mineur.</p>
```

```
<p th:if="${nom == 'Alice'}">Bienvenue, Alice !</p>
<p th:if="${nom != 'Alice'}">Bonjour, inconnu !</p>
```

Implémentation avec Spring Boot

- **La structure itérative :**
  - Syntaxe :
    - `th:each="varIter : ${collection}" th:text="${varIter}"`
  - Exemples :

```
@GetMapping("/etudiants")
fun liste(model: Model): String {
   val etudiants = listOf("Alice", "Bob", "Charlie")
   model.addAttribute("etudiants", etudiants)
   return "etudiants«
}
```

```
<ul>
    <li th:each="nom : ${etudiants}" th:text="${nom}">Nom Étudiant</li>
</ul>
```

Implémentation avec Spring Boot

- **La structure itérative :**
  - Il est également possible d'accéder à l'état de la boucle avec `iterStat`.
  - <u>Exemple :</u>

```
<li th:each="nom, iterStat : ${etudiants}">
  <span th:text="${iterStat.index}">0</span> - <span th:text="${nom}">Nom</span>
</li>
```

STRUCTURE DES FICHIERS DES TEMPLATES

- **Factoriser et optimiser le code de la vue avec le template :**
  - L'un des avantages des moteurs de template est qu'il est possible de décomposer les fichiers du template en modules que l'on nomme **fragments**.
  - Ainsi des éléments se répétant comme typiquement le *header* et le *footer* sont placés dans des fichiers séparés pour constituer des **fragments** que nous pouvons inclure dans les autres fichiers du template.
  - Enfin la construction des fichiers dont **`index.html`** prendra le fichier **`base.html`** pour établir leurs structures.

STRUCTURE DES FICHIERS DES TEMPLATES

- **La structure typique :**
  - Il est conseillé de suivre la structure suivante pour organiser ses templates

```
src/
└── main/
    └── resources/
        └── templates/
            ├── base.html
            ├── fragments/
            │   ├── header.html
            │   ├── navbar.html
            │   └── footer.html
            ├── index.html
```

LES FRAGMENTS

- **Syntaxe pour les fragments :**
  - Nous allons marquer nos fragments par un éléments syntaxique **Thymeleaf** directement dans la balise ouvrante du fragment.
  - Le **fragment** pourra recevoir des paramètres placés entre parenthèses.

```
<baliseOuvrante th:fragment="nomDuFragment(para1, para2, …)">
```

LES FRAGMENTS

- **Exemple de fragments :**

  - Exemple : `fragments/head.html`

```
<div xmlns:th="http://www.thymeleaf.org">
<head th:fragment="head(titre)">
    <meta charset="UTF-8">
    <title th:text="${titre}">Titre par défaut</title>
    <link rel="stylesheet" th:href="@{/css/style.css}">
</head>
</div>
```

  - **Remarque :** Le bloc n'est pas obligatoire, il nous permet juste d'indiquer à notre **IDE** qu'on utilise **Thymeleaf** afin qu'il fournisse l'auto-complétion.

LES FRAGMENTS

- **Exemple de fragments :**
  - Exemple : `fragments/navbar.html`

```
<div xmlns:th="http://www.thymeleaf.org">
<nav th:fragment="navbar">
     <a th:href="@{/}">Accueil</a> |
     <a th:href="@{/etudiants}">Étudiants</a>
</nav>
</div>
```

LES FRAGMENTS

- **Exemple de fragments :**
  - Exemple : `fragments/footer.html`

```
<div xmlns:th="http://www.thymeleaf.org">
<footer th:fragment="footer">
       <p>&copy; 2025 - Mon application</p>
</footer>
</div>
```

LE FICHIER DE BASE

- **Principe :**

  - Le fichier de base sera invoqué par les différentes pages de votre application afin qu'il puisse insérer leur contenu.

  - Le fichier de base est donc le squelette de base de toute page de votre site/application.

  - Le fichier de base va également directement invoquer des fragments comme celui du *footer* par exemple.

LE FICHIER DE BASE

- **Exemple d'un fichier de base.html**
  - Dans l'exemple ci-dessous le fichier de base importe 2 fragments (**navbar** et **footer**) et définit une section accueillant du contenu (**content**).

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org" lang="fr">
<head>
    <meta charset="UTF-8">
    <title th:text="${titre} ?: 'Mon site'">Mon site</title>
</head>
<body>
  <header th:replace="fragments/navbar :: navbar"></header>

  <main>
      <div th:replace=":: content"></div>
  </main>

  <footer th:replace="fragments/footer :: footer"></footer>
</body>
</html>
```

Fichier Exploitant le Fichier de Base

- **Exemple :**
  - Le fichier suivant indique qu'il utilise le fichier de base (`th:replace="base :: html"`) et définit un fragment qui sera à insérer (`th:fragment="content"`).

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org" th:replace="base :: html">

<div th:fragment="content">
  <h1>Liste des étudiants</h1>
  <ul>
    <li th:each="nom : ${etudiants}" th:text="${nom}"></li>
  </ul>
</div>

</html>
```

EXEMPLE D'IMPLÉMENTATION

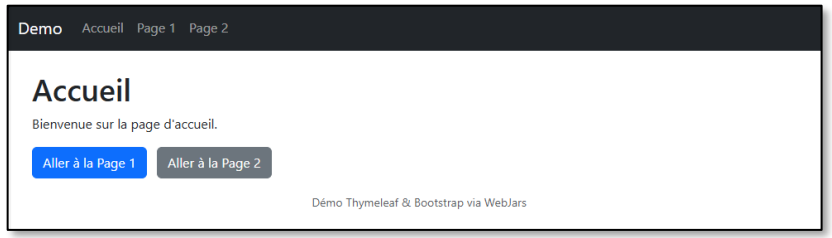
Création des fichiers du template & Navigation entre les pages

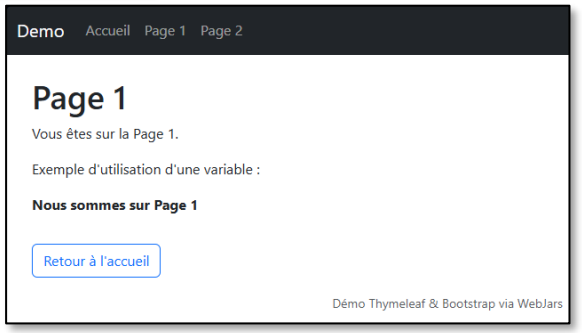
Exemple d'implémentation

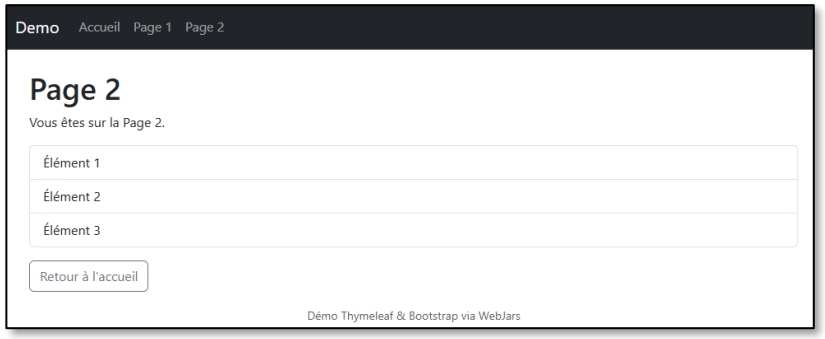
- **Objectifs de l'exemple :**
  - Initialiser le projet **Spring Boot**
  - Création du template minimal permettant la navigation
  - Importation de **Bootstrap** avec les **Webjars** et **Gradle**
  - Exploitation du **contrôleur** pour assurer la navigation entre les pages

Exemple d'implémentation

- **Résultat attendu :**
  - Une application avec 3 pages dont les éléments de navigation sont opérationnels.

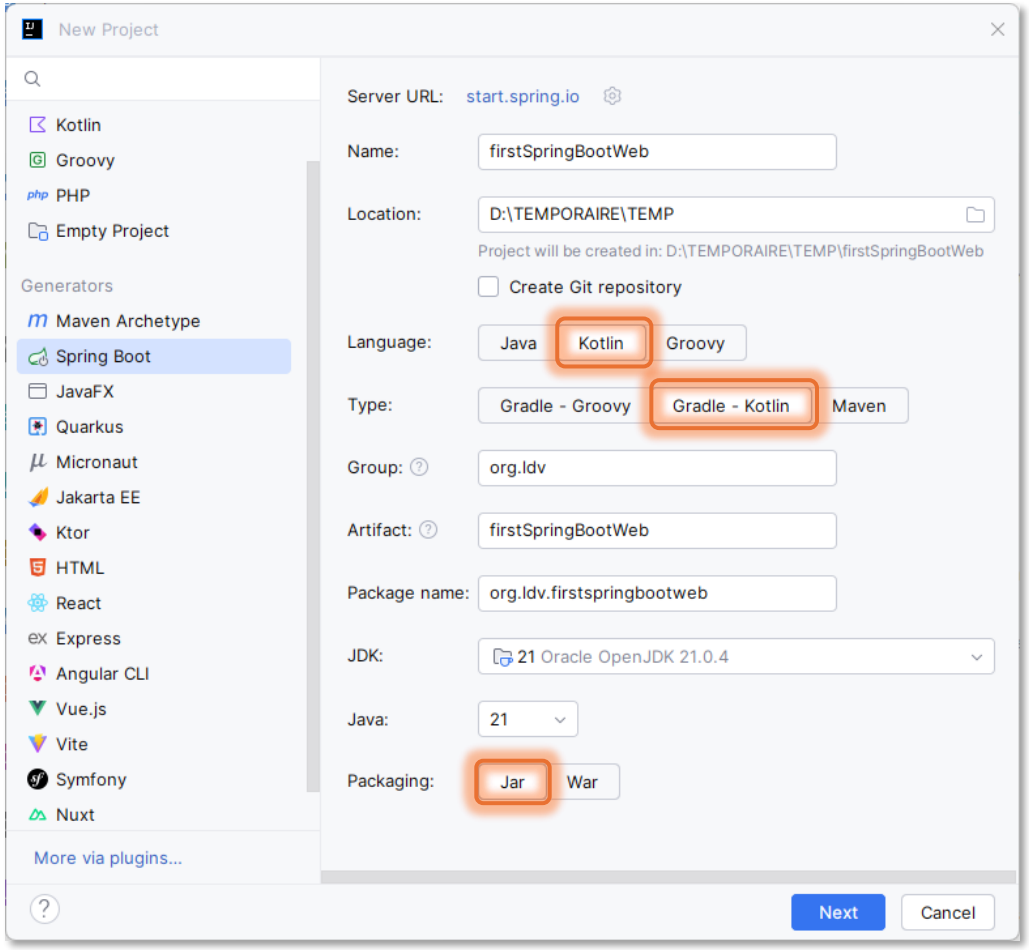


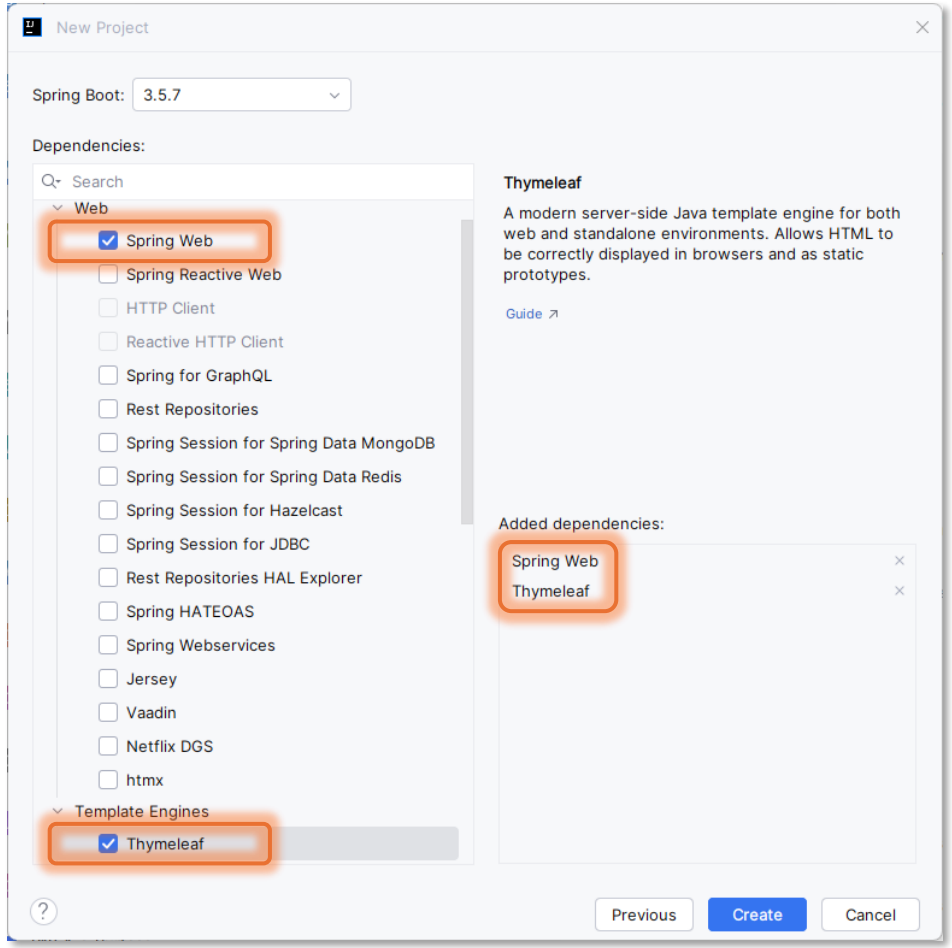




EXEMPLE D'IMPLÉMENTATION

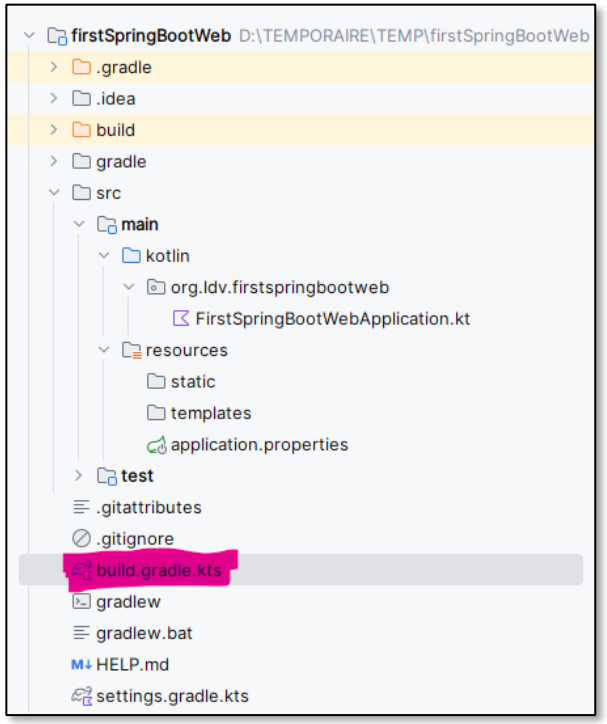
- **Etape 1 :** Initialisation du projet **SpringBoot**





EXEMPLE D'IMPLÉMENTATION

- **Etape 2 :** Ajout de **Bootstrap** via **WebJars**
  - Ouvrir **`build.gradle.kts`** et ajouter dans dependencies :
    - `implementation("org.webjars:bootstrap:5.3.8")`
    - `implementation("org.webjars:webjars-locator-core:0.52")`




```
dependencies { Add Starters...
    implementation("org.springframework.boot:spring-boot-starter-thymeleaf")
    implementation("org.springframework.boot:spring-boot-starter-web")
    implementation("com.fasterxml.jackson.module:jackson-module-kotlin")
    implementation("org.jetbrains.kotlin:kotlin-reflect")
    implementation("org.webjars:bootstrap:5.3.8")
    implementation("org.webjars:webjars-locator-core:0.52")
    testImplementation("org.springframework.boot:spring-boot-starter-test")
    testImplementation("org.jetbrains.kotlin:kotlin-test-junit5")
    testRuntimeOnly("org.junit.platform:junit-platform-launcher")
}
```

EXEMPLE D'IMPLÉMENTATION

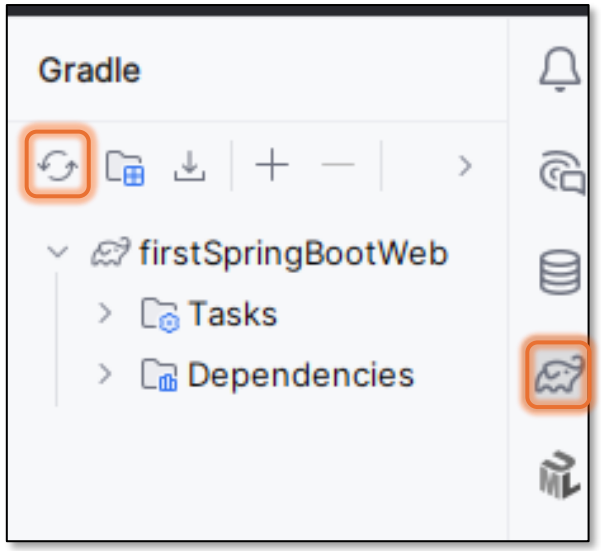
- **Option pour l'étape 2 :** Ajout des icônes **Bootstrap**
  - Si vous comptez utiliser les icônes de **Bootstrap** alors voici la ligne à ajouter :
    - `implementation("org.webjars.npm:bootstrap-icons:1.13.1")`

```
implementation("org.jetbrains.kotlin:kotlin-reflect")
implementation("org.webjars:bootstrap:5.3.8")
implementation("org.webjars.npm:bootstrap-icons:1.13.1")
implementation("org.webjars:webjars-locator-core:0.52")
```

  - Pour rappel vous trouverez les icônes à la page suivante : https://icons.getbootstrap.com/
  - En bas de la page vous trouverez les alternatives pour l'importation et des exemples d'utilisation.

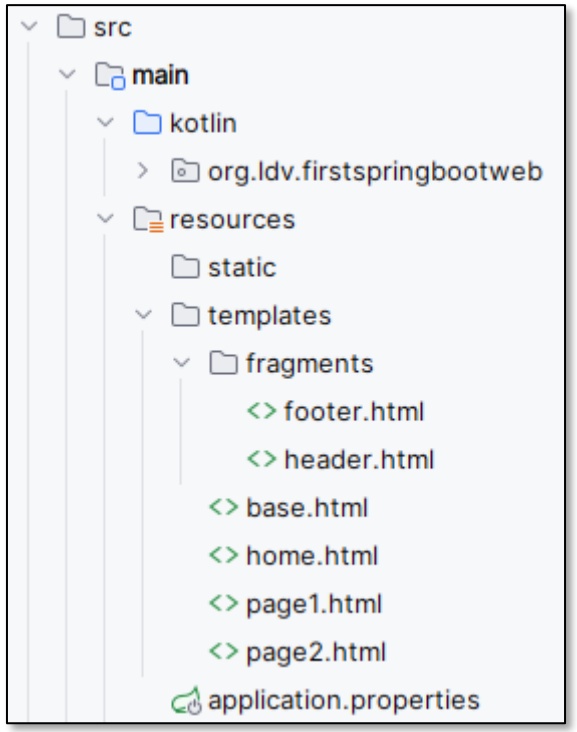
Exemple d'implémentation

- **Etape 2 :** Ajout de **Bootstrap** via **WebJars**
    - Resynchroniser **Gradle** :



Exemple d'implémentation

- **Etape 3 :** Création du template
  - Création des fichiers et dossiers du template :
    - Le dossier **`templates`** a été créé automatiquement et accueille la structure de notre template :
      - Le dossier **`fragments`** qui contient :
        - **`footer.html`**
        - **`header.html`**
      - Les fichiers :
        - **`base.html`**
        - **`home.html`**
        - **`page1.html`**
        - **`page2.html`**



EXEMPLE D'IMPLÉMENTATION

- **Etape 3 :** Création du template
  - Code du fichier **`base.html`** :

```html
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org" lang="fr">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">

    <title th:text="${titre} ?: 'Démo Thymeleaf'">Démo Thymeleaf</title>

    <!-- Bootstrap via WebJars + Bootstrap icône si besoin -->
    <link rel="stylesheet" th:href="@{/webjars/bootstrap/css/bootstrap.min.css}">
    <link rel="stylesheet" th:href="@{/webjars/bootstrap-icons/font/bootstrap-icons.css}">
</head>
<body>

<!-- header -->
<header th:replace="fragments/header :: header"></header>

<!-- Contenu principal -->
<main class="container mt-4">
    <!-- Contenu spécifique des pages -->
    <div th:replace=":: content"></div>
</main>

<!-- Footer -->
<footer th:replace="fragments/footer :: footer"></footer>

<!-- JS Bootstrap via WebJars -->
<script th:src="@{/webjars/bootstrap/js/bootstrap.bundle.min.js}"></script>
</body>
</html>
```

Exemple d'implémentation

- **Etape 3 :** Création du template
  - Code du fichier **`footer.html`** :

```
<div xmlns:th="http://www.thymeleaf.org">
<footer th:fragment="footer" class="text-center text-muted py-3">
    <small>Démo Thymeleaf & Bootstrap via WebJars</small>
</footer>
</div>
```

EXEMPLE D'IMPLÉMENTATION

- **Etape 3 :** Création du template
  - Code du fichier **`header.html`** :

```
<div xmlns:th="http://www.thymeleaf.org">
<header th:fragment="header">
<nav class="navbar navbar-expand-lg navbar-dark bg-dark">
    <div class="container-fluid">
        <a class="navbar-brand" th:href="@{/}">Demo</a>
        <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target="#navbarNav">
            <span class="navbar-toggler-icon"></span>
        </button>

        <div class="collapse navbar-collapse" id="navbarNav">
            <ul class="navbar-nav me-auto">
                <li class="nav-item">
                    <a class="nav-link" th:href="@{/}">Accueil</a>
                </li>
                <li class="nav-item">
                    <a class="nav-link" th:href="@{/page1}">Page 1</a>
                </li>
                <li class="nav-item">
                    <a class="nav-link" th:href="@{/page2}">Page 2</a>
                </li>
            </ul>
        </div>
    </div>
</nav>
</header>
</div>
```

EXEMPLE D'IMPLÉMENTATION

- **Etape 3 :** Création du template
  - Code du fichier **`home.html`** :

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org" lang="fr"
      th:replace="base :: html">
<div th:fragment="content">
    <h1 th:text="${titre}">Accueil</h1>
    <p th:text="${message}">Texte d'accueil.</p>

    <div class="mt-3">
        <a class="btn btn-primary" th:href="@{/page1}">Aller à la Page 1</a>
        <a class="btn btn-secondary ms-2" th:href="@{/page2}">Aller à la Page 2</a>
    </div>
</div>
</html>
```

Exemple d'implémentation

- **Etape 3 :** Création du template
  - Code du fichier **`page1.html`** :

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org" lang="fr"
      th:replace="base :: html">
<div th:fragment="content">
    <h1 th:text="${titre}">Page 1</h1>
    <p th:text="${message}">Contenu de la page 1.</p>

    <p>Exemple d'utilisation d'une variable :</p>
    <p><strong th:text="'Nous sommes sur ' + ${titre}"></strong></p>

    <a class="btn btn-outline-primary mt-3" th:href="@{/}">Retour à l'accueil</a>
</div>
</html>
```

EXEMPLE D'IMPLÉMENTATION

- **Etape 3 :** Création du template
  - Code du fichier **`page2.html`** :

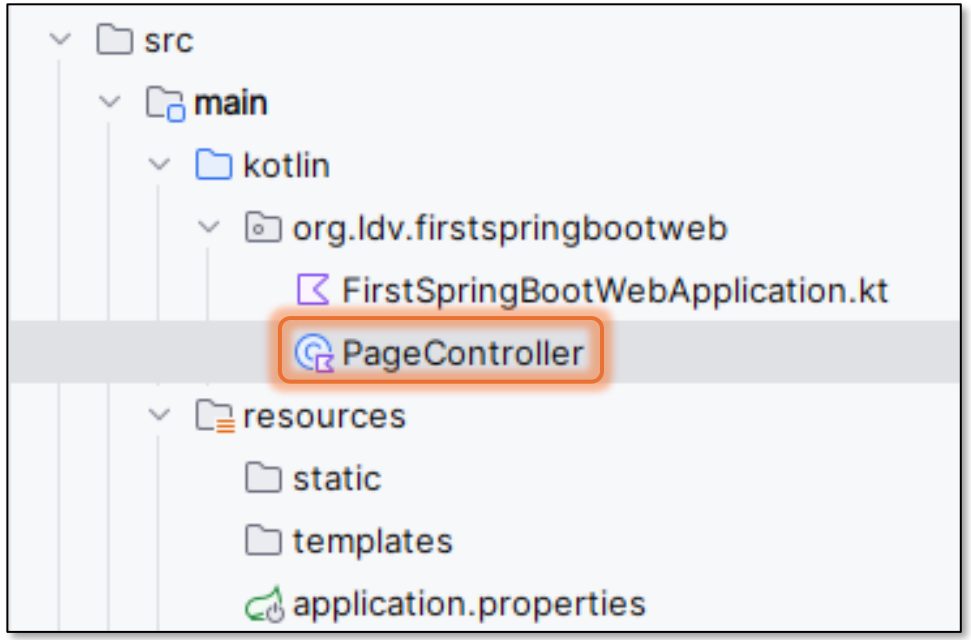
```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org" lang="fr"
      th:replace="base :: html">
<div th:fragment="content">
    <h1 th:text="${titre}">Page 2</h1>
    <p th:text="${message}">Contenu de la page 2.</p>

    <ul class="list-group mt-3">
        <li class="list-group-item">Élément 1</li>
        <li class="list-group-item">Élément 2</li>
        <li class="list-group-item">Élément 3</li>
    </ul>

    <a class="btn btn-outline-secondary mt-3" th:href="@{/}">Retour à l'accueil</a>
</div>
</html>
```

Exemple d'implémentation

- **Etape 4 :** Création du contrôleur de navigation
  - Création du fichier : **`PageController.kt`**



Exemple d'implémentation

- **Etape 4 :** Création du contrôleur de navigation
  - Insertion du code du contrôleur :

```
package org.ldv.firstspringbootweb

import org.springframework.stereotype.Controller
import org.springframework.ui.Model
import org.springframework.web.bind.annotation.GetMapping

@Controller
class PageController {

    @GetMapping("/")
    fun home(model: Model): String {
        model.addAttribute("titre", "Accueil")
        model.addAttribute("message", "Bienvenue sur la page d'accueil.")
        return "home"
    }

    @GetMapping("/page1")
    fun page1(model: Model): String {
        model.addAttribute("titre", "Page 1")
        model.addAttribute("message", "Vous êtes sur la Page 1.")
        return "page1"
    }

    @GetMapping("/page2")
    fun page2(model: Model): String {
        model.addAttribute("titre", "Page 2")
        model.addAttribute("message", "Vous êtes sur la Page 2.")
        return "page2"
    }
}
```

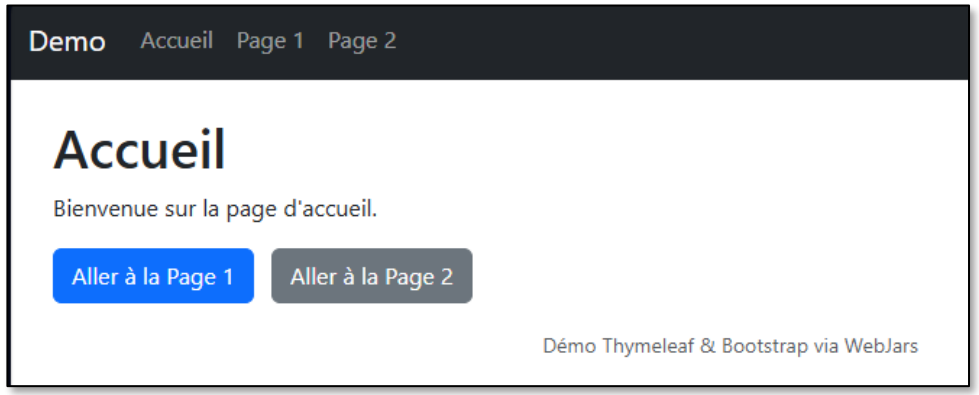
Exemple d'implémentation

- **Test :**
  - **L'exécution du projet doit maintenant être opérationnel :**

```
  .   ____          _            __ _ _
 /\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
 \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
  '  |____| .__|_| |_|_| |_\__, | / / / /
 =========|_|==============|___/=/_/_/_/

 :: Spring Boot ::                (v3.5.7)

2025-11-10T07:03:34.293+01:00  INFO 29188 --- [firstSpringBootWeb] [           main] o.l.f.FirstSpringBootWebApplicationKt    : Starting FirstSpringBootWebApplicationKt using Java
 21.0.4 with PID 29188 (D:\TEMPORAIRE\TEMP\firstSpringBootWeb\build\classes\kotlin\main started by ljules in D:\TEMPORAIRE\TEMP\firstSpringBootWeb)
2025-11-10T07:03:34.296+01:00  INFO 29188 --- [firstSpringBootWeb] [           main] o.l.f.FirstSpringBootWebApplicationKt    : No active profile set, falling back to 1 default
 profile: "default"
2025-11-10T07:03:35.113+01:00  INFO 29188 --- [firstSpringBootWeb] [           main] o.s.b.w.embedded.tomcat.TomcatWebServer  : Tomcat initialized with port 8080 (http)
2025-11-10T07:03:35.125+01:00  INFO 29188 --- [firstSpringBootWeb] [           main] o.apache.catalina.core.StandardService   : Starting service [Tomcat]
2025-11-10T07:03:35.126+01:00  INFO 29188 --- [firstSpringBootWeb] [           main] o.apache.catalina.core.StandardEngine    : Starting Servlet engine: [Apache Tomcat/10.1.48]
2025-11-10T07:03:35.170+01:00  INFO 29188 --- [firstSpringBootWeb] [           main] o.a.c.c.C.[Tomcat].[localhost].[/]       : Initializing Spring embedded WebApplicationContext
2025-11-10T07:03:35.170+01:00  INFO 29188 --- [firstSpringBootWeb] [           main] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: initialization completed
 in 815 ms
2025-11-10T07:03:35.687+01:00  INFO 29188 --- [firstSpringBootWeb] [           main] o.s.b.w.embedded.tomcat.TomcatWebServer  : Tomcat started on port 8080 (http) with context path
 '/'
2025-11-10T07:03:35.693+01:00  INFO 29188 --- [firstSpringBootWeb] [           main] o.l.f.FirstSpringBootWebApplicationKt    : Started FirstSpringBootWebApplicationKt in 1.746
 seconds (process running for 2.25)
```



Référence des Eléments Statiques

Références sur les images & éléments de styles

Référence des Eléments Statiques

- Dans un projet **Spring Boot**, les ressources statiques (**CSS**, **JS**, **images**) doivent être placées dans un des répertoires suivants (**Spring** les reconnaît automatiquement) :

```
src/
└── main/
    ├── kotlin/com/example/demo/
    ├── resources/
    │   ├── static/
    │   │   ├── css/style.css
    │   │   ├── js/script.js
    │   │   └── images/
    │   │       ├── logo.png
    │   │       ├── hero-banner.jpg
    │   │       └── icones/
    │   │           └── user.png
    │   └── templates/
    │       ├── base.html
    │       ├── home.html
    │       └── fragments/
    │           └── navbar.html
```

Référence des Eléments Statiques

- **Accès aux images dans les templates Thymeleaf**
  - Utiliser `th:src` pour les images
  - Exemple :

```
<img th:src="@{/images/logo.png}" alt="Logo du site">
```

Référence des Eléments Statiques

- **Bonnes pratiques :**

Bonnes pratiques	Explications
Noms en minuscules et sans espace	Exemple : `hero-banner.jpg`, `logo_footer.png`
Sous-dossiers par type ou usage	`/images/icones/`, `/images/produits/`, `/images/users/`
Formats adaptés	**PNG** pour icônes, **JPG** pour photos, **SVG** pour logos vectoriels
Taille optimisée	Alléger les images (TinyPNG, Squoosh…) pour accélérer le chargement
Utiliser `alt` toujours	Accessibilité + SEO

Référence des Eléments Statiques

- **Accès au style CSS**
  - Utilisation également de `th:src`
  - Dans l'exemple `style.css` est le nom de votre fichier **CSS** qui est placé dans le dossier `src/main/ressources/static/css`

  - Exemple :

```
<link rel="stylesheet" th:href="@{/css/style.css}">
```

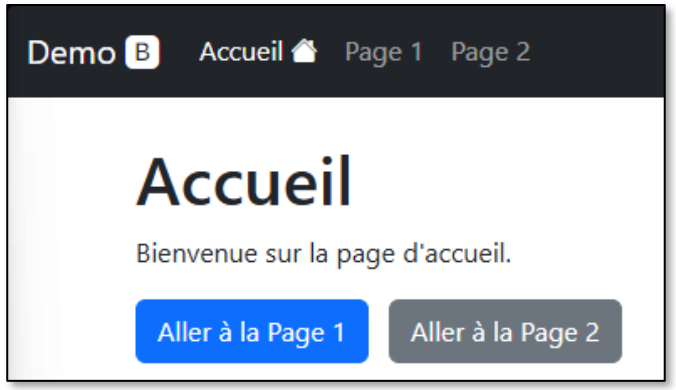
Améliorations & Autres options

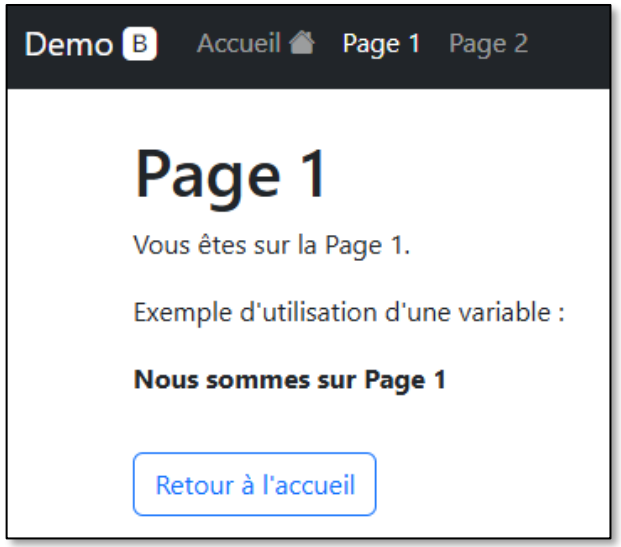
Gestion de la class active de Bootstrap

Améliorations & Autres Options

- **Gestion de la classe active :**
  - Afin de mettre en évidence la page active, **Bootstrap** propose une classe **`active`** pour mettre en avant le lien actif du lien.

  - <u>Illustration :</u>





Améliorations & Autres Options

- **Gestion de la classe active :**
  - Pour implémenter cette fonctionnalité il faut communiquer la page courante à la vue.
  - Nous allons enrichir les méthodes associées aux templates afin qu'elles rajoutent un identifiant unique au contexte de données transmis à la page associée.

```
@GetMapping( ...value = "/")
open fun home(model: Model): String {
    model.addAttribute( attributeName = "titre",  attributeValue = "Accueil")
    model.addAttribute( attributeName = "message",  attributeValue = "Bienvenue sur la page d'accueil.")
    model.addAttribute( attributeName = "activePage",  attributeValue = "home")
    return "home"
}
```

Améliorations & Autres Options

- **Gestion de la classe active :**

```
@GetMapping(...value = "/page1")
open fun page1(model: Model): String {
    model.addAttribute( attributeName = "titre",  attributeValue = "Page 1")
    model.addAttribute( attributeName = "message",  attributeValue = "Vous êtes sur la Page 1.")
    model.addAttribute( attributeName = "activePage",  attributeValue = "page1")
    return "page1"
}

@GetMapping(...value = "/page2")
open fun page2(model: Model): String {
    model.addAttribute( attributeName = "titre",  attributeValue = "Page 2")
    model.addAttribute( attributeName = "message",  attributeValue = "Vous êtes sur la Page 2.")
    model.addAttribute( attributeName = "activePage",  attributeValue = "page2")
    return "page2"
}
```

Améliorations & Autres Options

- **Gestion de la classe active :**

  - Les différentes pages appelées (*home*, *page 1* et *page 2*) récupèrent leur identifiant grâce au contexte qui leur est transmis.

  - Ce contexte est partagé par les différents fragments et/ou fichier de base.

  - Par conséquent nous pouvons utiliser directement la variable **`activePage`** dans le fragment **`header`**. Nous allons utiliser l'instruction `th:classappend` pour ajouter la classe active pour l'élément concerné.

Améliorations & Autres Options

- **Gestion de la classe active :**
  - Voici le code implémenté pour l'entrée de la page d'accueil :

```
<li class="nav-item">
    <a  class="nav-link"
        th:href="@{/}"
        th:classappend="${activePage} == 'home' ? ' active'"
        >Accueil <i class="bi bi-house-fill"></i>
    </a>
</li>
```

Améliorations & Autres Options

- **Gestion de la classe active :**

  - Voici le code complet du fichier **`header.html`** de notre exemple :

```
<div class="collapse navbar-collapse" id="navbarNav">
    <ul class="navbar-nav me-auto">
        <li class="nav-item">
            <a  class="nav-link"
                th:href="@{/}"
                th:classappend="${activePage} == 'home' ? ' active'"
                >Accueil <i class="bi bi-house-fill"></i>
            </a>
        </li>
        <li class="nav-item">
            <a  class="nav-link"
                th:href="@{/page1}"
                th:classappend="${activePage} == 'page1' ? ' active'"
                >Page 1
            </a>
        </li>
        <li class="nav-item">
            <a  class="nav-link"
                th:href="@{/page2}"
                th:classappend="${activePage} == 'page2' ? ' active'"
                >Page 2
            </a>
        </li>
    </ul>
</div>
```